

Comparing Segmentation Methods with Different Base Classifiers

Roberto Angelo Fernandes Santos^{1,2} and Roberto Souto Maior de Barros¹

¹Centro de Informática, Universidade Federal de Pernambuco, 50.740-560, Recife-PE, Brazil.

²Faculdade Impacta de Tecnologia, 04.043-300, São Paulo-SP, Brazil.

Abstract - Large Companies often use statistical models and artificial intelligence to predict several kinds of risks. In general, only one statistical model or a neural network technique is used to predict these risks. It is not so uncommon for some companies to use segmentation to define subpopulation splits for the development of specific predictors to improve decision accuracy. However, the use of segmentation usually depends on the experience of the specialist. The basic premise of this paper is to compare three kinds of segmentation methods, crossed with three different base classifiers, and using five datasets from the UCI Repository. The segmentation methods are: NNTree - a Neural Network Tree based method, the traditional segmentation using Information Gain to split, and an experimental method called FBTSeg. Results show that blending classifiers using segmentation is a viable option to improve results of known and traditional techniques such as statistic regressions and neural networks.

Keywords: Classification methods; Segmentation; Data splitting; Machine learning.

1 Introduction

The capacity for predicting events is the key element in the decision support processes for modern corporations. The results of decision models based on data are dependent on the quantity and quality of the information available. However, improvements in prediction techniques are also important for better accuracy, as several academic and practical initiatives from different areas show [1] [2] [3]. Moreover, this search goes beyond a single predictive model and is also progressing towards the combination of several predictors, both from the same and from different techniques. The literature shows that combining techniques using different sampling methods or using predictors of distinct characteristics usually offers more predictive solutions [4] [5] [6] [7] [8] [9] [10].

Combining predictors is not simple and its complexity is derived from a number of factors inherent to such combinations. Some questions that illustrate this complexity are: What is the best combination method for a specific problem? What techniques should be combined? How many predictors are needed to maximize performance? The fact is

there are no simple or exact answers to those questions and most of them can only be answered based on the specific application constraints, the expertise of the developers, and results of practical experiments.

Segmentation (decomposition) [11] is one of the mostly used methods for the combination of predictive techniques and proposes the transformation of a complex problem in several less complex ones as the means to achieve the best solution to the original problem. In data mining, segmentation is used to obtain subgroups where different learning techniques may be applied, i.e. specialized predictors are applied to subsets of the original data set.

For a long time, the techniques used to find good segmentation structures were based on subjective strategies and on previous knowledge of the problem. One of these subjective techniques is separation of the data in two groups, one with a number of attributes with their values filled in, and the other with many missing values in these attributes, since this is a common situation in credit scoring problems [12]. It was even suggested that it might not be possible to create the segmentation structure without using subjective knowledge. However, methods and algorithms to find adequate segmentation structures have already been proposed. They usually represent the segmentation structure as a tree and use exhaustive or semi-exhaustive search to choose the best segmentation strategy [11].

This paper compares the performance of three different segmentation methods to build model trees. We also briefly describe a new semi-exhaustive method called FBTSeg, formerly RISKSEG [20], that implements an algorithm to optimize and build a binary decision tree with regression models in each leaf. To do this comparison we used five datasets from the UCI Repository [13].

2 Review of Used Techniques

In this section we briefly review the techniques used in the experiments, both single classifiers and their combination using segmentation.

Binary Logistic Regression is a traditional technique of statistic inference [14] that relates a dichotomic target (*good*

or *bad*, 1 or 0, *fraud* or *not fraud*, etc.) to a set of predictive input variables $X_1, X_2, \dots, X_n$, where Y is a random variable with two possible values, 0 (zero) and 1 (one). Its result is an estimation of the probability of Y belonging to one of the two categories, based on the observed values of the predicting variables. In this paper, the estimation of $Y = 1$ will be used. The dependent variable is the logarithm of the ratio of probabilities of the two possible results of an output variable, $\log [p / (1-p)]$. We used the widespread maximum likelihood method for estimating the binary response, having its solution found through Newton-Raphson's iterative process [15].

Linear Discriminant Analysis estimates the output variable based on the observed values of predicting variables and is usually applied when the dependent variable is numeric and continuous. Errors are assumed to have normal distribution [16]. In this paper, we use this technique to estimate the value of a dichotomic variable (0 or 1).

Multilayer Perceptron Neural Network (MLP) [17] is a computational intelligence technique that has excellent generalization capabilities, is simple to implement, and is commonly used in classification problems. In our experiments, we tested MLP with the backpropagation and Levenberg-Marquardt algorithms [17].

Segmentation is the division of the available dataset in subpopulations and the development of a prediction model for each segment. Therefore, each example is predicted according to the specific formula (model) that it belongs. The segmentation provides the most appropriate use of available information and is highly recommended when there is a large number of missing data in dataset samples. The segmentation is also recommended for capturing risk relationships among the subpopulations with different profiles – this way the specialized models in that data sample are able to explore their peculiarities more efficiently [12].

As in decision trees, one of the main criteria used for segmentation is Information Gain [18]. Decision trees aim to separate the population in disjoint subsets that are homogeneous regarding the behavior of the output variable, because distinct behaviors regarding the output variable might lead to large variations in the estimation of the regression parameters. Moreover, distinct behaviors in separate groups are to be used as the basis for the creation of multiple classifiers in the FBTSeg method.

An NNTree [19] “is a decision tree, where each non-terminal node contains a neural network”. Its segmentation uses a multilayer perceptron neural network to design the classification tree. The method recursively divides the training set trying to create leaves that contain elements from a single class – each non-terminal node has elements from more than one class and uses a neural network to separate them. Even though the original implementation only uses neural networks,

we have adapted the algorithm to also support both logistic regression and linear regression in the non-terminal nodes.

2.1 Find Best Tree Segmentation (FBTSeg)

The FBTSeg method [20] rationale is to try to find a set of segmented models with better performance using a binary tree built from divisions using the attribute classes. This method uses a specific algorithm for the selection of n attributes to test all combinations for each candidate node, where n is a parameter. In other words, after the attribute selection, each n attribute class is used to split and the best segmentation is chosen to improve the accuracy of the model for this node. Every node generates two leaves with their specific models which, when combined, generate better results than the upper node model. In summary, the best option to split each node sample is chosen. It is important to choose a specific (or the best) metric for optimizing the results of the problem in hand, such as classification error, mean squared error, KS2 [21], ROC [22], and others. To determine the best segmentation for a node split, it is necessary to join the scores of the corresponding pair of leaves, from the set of possibilities, and compare their results.

The FBTSeg method is prone to overfitting, a known problem in other supervised learning algorithms. So, it is important to create a validation dataset to apply a cross-validation between the training and validation datasets. Then the training dataset is just used to train the predictors, while the validation dataset is used to compare the scores among the segments and for determining the best segmentation possible. There is also a parameter to set the minimal number of observations to train and validate a segmented model, aimed at ensuring a minimal set to make a representative model in each leaf. Of course, its value must be established taking into account the sample size available to develop the models.

The implementation of models in subpopulations is similar to making a model with the main predictive variables combined with the variables resulting from interactions among the main variables and a selected variable [12] – see equations 1 to 5. Through the selection of good segmentations it is possible to add the most important characteristics found in the interactions of the variables. Thus, in the FBTSeg segmentation, for each terminal node that might be segmented, the possible interactions of the model variables are analyzed. The implementation algorithm builds a tree model based on the best candidate variables, those that have the best interactions with the other variables according to the chosen metric. This scheme avoids the exhaustive search for the best segmentation and permits optimizing the final result based on some chosen metric. Because it is easier to understand, we illustrate the method using a statistical linear regression with binary variables (values 0 or 1), but our implementation used logistic regression with categorical variables.

Let x_1, x_2, x_3 , and x_4 be random variables, Y a dependent variable (target), $\mu_i, \beta_i, \lambda_i, \alpha_i, \omega_i, \theta_i$ the constant weights of the equations, and ε_i the error factor. We defined a simple linear regression model that only considers the main effects of the variables – no interaction term was included in the equation for parameters estimation.

$$Y = \mu_1 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \varepsilon_1 \quad (1)$$

Including interaction terms $x_i x_j$ (with i fixed and j varying) is equivalent to segment the data by x_i and adjust separate regressions for each subgroup. For example:

$$Y = \mu_2 + \lambda_1 x_1 + \lambda_2 x_2 + \lambda_3 x_3 + \lambda_4 x_4 + \lambda_{12} x_1 x_2 + \lambda_{13} x_1 x_3 + \lambda_{14} x_1 x_4 + \varepsilon_2 \quad (2)$$

In this example (2), we capture the effects of the interactions between x_1 and the other variables of the model. This is similar to segmenting the data based on the values of x_1 and adjusting different regressions to the generated subgroups. Similarly, other three models can be defined:

$$Y = \mu_3 + \alpha_1 x_1 + \alpha_2 x_2 + \alpha_3 x_3 + \alpha_4 x_4 + \alpha_{12} x_1 x_2 + \alpha_{23} x_2 x_3 + \alpha_{24} x_2 x_4 + \varepsilon_3 \quad (3)$$

$$Y = \mu_4 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \omega_4 x_4 + \omega_{13} x_1 x_3 + \omega_{23} x_2 x_3 + \omega_{34} x_3 x_4 + \varepsilon_4 \quad (4)$$

$$Y = \mu_5 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4 + \theta_{14} x_1 x_4 + \theta_{24} x_2 x_4 + \theta_{34} x_3 x_4 + \varepsilon_5 \quad (5)$$

These three models are equivalent to segmentations based on x_2, x_3 , and x_4 , respectively. So, adjusting the four regressions makes it possible to choose the best regression according to some performance measure D , i.e. the best segmentation variable. It is also necessary to choose the splitting rule, specifically, a category or a numeric interval.

The FBTSeg search is *not* the semi-exhaustive search because it does *not* test all the possible split rules of all variables. The number of splits is limited by the number of candidate variables categories. This method also permits adjusting the regressions using a subset of the training set, which makes it faster when the training set is very large.

Algorithm 1 gives a possible implementation of FBTSeg showing the search for possible segmentations. The first loop selects the best variable and the second chooses the category of this variable that may be used to split the data. So, after selecting the variable (*Var*) with the best regression results in the interaction with the other variables (factorial), all the possible binary combinations of this variable categories are tested to find the best category (*Cat*) and generate the best pair of segments, one for "*Var = Cat*" and another for "*Var $\neq$ Cat*". After the segmentation, the predictive performance of the suggested segments is compared to that of the upper node. The segmentation is confirmed if it improves the results of the upper node. As in decision trees, the segmentation continues

until some stop criterion is reached. The final result is a set of models generated from disjoint subsets of the training data.

3 Experimental Design and Results

We run experiments with 5 (five) datasets of the UCI Machine Learning Repository [13] to compare the methods. The chosen datasets were the ones with a dichotomic target and at least 1,000 records. These are:

- *Chess* – Contains 3,196 records with information about chess matches. The peaces' positions are represented by 36 categorical variables and the target is dichotomic (0=white wins and 1=black wins).
- *German* – Credit base with 1,000 records. It contains the most common target variable in credit risk studies for the representation of good and bad payers (1=Good and 2=Bad). The input variables are mostly social-economic attributes, 20 in total, seven of them are numeric and 13 are categorical.
- *Magic Gamma Telescope* – This dataset simulates the registration of high energy gamma particles in a ground-based telescope using the imaging technique. There are 19,020 records, 10 numeric variables, and a dichotomic target with values "g" (gamma) and "h" (hadron), respectively transformed to 0 (zero) and 1 (one).
- *Adult* – Contains information of the 1994 American Census database and aims to determine whether a person makes over US\$50,000 a year (1=yes) or not (0=no). There are 48,842 records and 14 variables, six of them are numeric and eight are categorical.
- *Spambase* – Contains 4,601 records and 57 numeric variables with information on the characteristics of e-mails. Its purpose is to build a spam filter, deciding whether each message is a spam (1=yes) or not (0=no).

In the preprocessing of these datasets, no categorical variable was grouped. Thus, their original values were used to generate their equivalent binary codifications, transforming each categorical variable (with n categories) in n binary variables (with values 0 or 1). On the other hand, all numeric variables were normalized with continuous values in the [0, 1] interval. However, because the implementation of the FBTSeg segmentation and also of the traditional segmentation by information gain both expect categorical variables to split the nodes, it was also necessary to categorize all numeric variables and, so, we decided to create four categories, each with approximately 25% of the data. Nevertheless, the generated categorical variables were only used in the segmentations – the training used their normalized numeric versions.

We also used the k -fold Cross Validation technique in the experiments and the chosen value for k was 10. This is a fairly common technique in experiments involving classifier

N – Training set of a given terminal node
 S_1, S_2 – Subsets of N generated by the segmentation
 p – Number of input variables of N
 $X_1, X_2, \dots, X_p$ – Discrete input variables of N
 w – Local variable that stores the number of categories of each discrete variable
 $C[w]$ – Local array that stores the values of each discrete variable
 y – Target variable (dependent)
 $fMet()$ – Performance Function (larger values are considered to be better in this example)
 $fMerge()$ – Merge function to join the results of the models
 Var – Variable chosen to test its categories
 Cat – Category of Var
 $rMod$ – Answers of the factorial models
 $rMod_0$ – Best answers of the models
 $rMod_1, rMod_2$ – Answers of the models of the candidate segments
 $rMod_{12}$ – Joint answer of the models of the candidate segments

Input: N ;

```

 $rMod_0 \leftarrow \emptyset$ ; (Best answers initialization)
For  $i = 1$  to  $p$  do { (Loop over all the input variables)
   $rMod \leftarrow$  Regression answers  $y \sim \sum_{j=1}^P X_j + \sum_{j=1, j \neq i}^P X_j X_i$ , with data from  $N$ ; (Trains model with interactions)

  If  $fMet(rMod) > fMet(rMod_0)$  { (If the model improves the result)
     $rMod_0 \leftarrow rMod$ ; (Saves the best answers of the model)
     $Var \leftarrow X_i$ ; (Saves the chosen segmentation variable)
  };
};

 $rMod_0 \leftarrow \emptyset$ ; (Best answers re-initialization)
 $rMod_{12} \leftarrow \emptyset$ ; (Mod12 answers set initialization)
 $C[w] \leftarrow$  Category values of  $Var$ ;
 $w \leftarrow$  Number of categories of  $Var$ ;

For  $i = 1$  to  $w$  do { (Loop over the categories of  $Var$ )
   $S_1 \leftarrow$  Subset of  $N$ , such as  $Var = C[w]$ ; (Creates the two training subsets)
   $S_2 \leftarrow$  Subset of  $N$ , such as  $Var \neq C[w]$ ;

   $rMod_1 \leftarrow$  Regression answers  $y \sim \sum_{j=1}^P X_j$ , with data from  $S_1$ ; (Trains the models of the two subsets)

   $rMod_2 \leftarrow$  Regression answers  $y \sim \sum_{j=1}^P X_j$ , with data from  $S_2$ ;

   $rMod_{12} \leftarrow rMod_1 \cup rMod_2$ ; (Joins the answers of the models of the segments)
  If  $fMet(rMod_{12}) > fMet(rMod_0)$  { (If the segmented models improve the result)
     $rMod_0 \leftarrow rMod_{12}$ ; (Saves the best answers of the models)
     $Cat \leftarrow C[w]$ ; (Saves the chosen segmentation category)
  };
};

Output:  $Var, Cat$ ; (Best segmentation variable and category)

```

Algorithm 1 – FBTSeg algorithm that chooses the best segmentation.

combination [23]. Therefore, the datasets were separated in 10 parts, each with 10% of the data, and, in each experiment, one of them was the testing set. The rest of the data was used for training (65%) and validation (35%).

The simulations were implemented in such a way that each single classifier was used as base classifier in the comparison with the segmentation methods.

The main difference regarding the three segmentation methods is the way the segment splits are chosen. NNTree originally uses an MLP neural network to split the training dataset [19], but we also implemented adaptations for using linear and logistic regressions as well. These three prediction techniques were also used with the other two segmentation methods and, for each of them, the behavior of the combination methods for the generation of multiple classifiers

was observed and the results measured using the classification error rate. We named “Simple” the classifications using each of the three chosen techniques as single base classifier, and “TradSeg” the segmentations by information gain.

3.1 Defining Experiment Parameters

In both regressions, it was not necessary to choose variables or parameters. The choice of parameters for training the MLP was based on the test for the best number of neurons in the intermediate level and the best training algorithm. For each data set, we tested all the possible combinations of the two training algorithms (backpropagation and Levenberg-Marquardt) with 3, 10, and 20 neurons in the intermediate layer. The chosen activation function was logistic function having the minimum squared error on the cross-validation set as the training stopping criterion. We used the standard learning rate of SAS Enterprise Miner [24], version 4.0, which cannot be modified in this version of the software. We then chose the best configuration for the single classifier and for the segmentations.

We decided to use similar parameters in the training of the three methods as much as possible, as the means to focus on the way the dataset splits are done. For the three segmentation methods, we used 2 (two) as the maximum depth of the tree because, in preliminary tests, we observed that the best segmentations were in these two levels. For the minimum size of a leaf node we chose 5%, a size we judged appropriate, considering that all datasets have at least 1,000 examples. The metric used to evaluate the results of the experiments was the classification error. Our implementation

of the segmentation methods used a validation dataset to determine if there was a gain (or not) in each segmentation possible. The validation dataset used was 35% of the training set and the same dataset was used in the validation of the MLP neural network.

3.2 Results

Table 1 presents the average classification errors (percentages) of the three single classifiers crossed with the three segmentation methods in the five data sets tested. The comparison is always based on the control classifier (Simple) of each technique. We used the t-Student test for paired data, with significance level of 5%. Average errors that are smaller than that of the respective base classifier are written in bold. Smaller and statistically different values are marked with an asterisk (*). Greater and significantly worse values are written in italics. We considered the best classifier the one with the smaller average error, as long as it is significantly better than the base classifier.

In the Chess dataset, using MLP, the three segmentations presented better average results, and statistically significant, when compared to the base classifier, and the FBTSeg method was considerably better than the second best result (TradSeg). Using linear regression, again, all three segmentations improved the results of the base classifier considerably, but the best results were obtained with the TradSeg method and its results were significantly better than the others. Using logistic regression, only the FBTSeg method presented significantly better results than the base classifier – TradSeg was only marginally better than the base classifier.

Table 1 – Average errors and confidence intervals.

Dataset	Technique	Simple	TradSeg	NNTree	FBTSeg	Best
Chess	MLP	4.07 ±1.05	1.09 ±0.42*	2.81 ±0.59*	0.35 ±0.14*	FBTSeg
	Linear Reg.	8.57 ±1.64	2.28 ±0.55*	3.18 ±0.68*	3.25 ±0.77*	TradSeg
	Logistic Reg.	2.60 ±0.73	2.25 ±0.83	2.63 ±0.51	1.02 ±0.27*	FBTSeg
German	MLP	25.20 ±1.79	25.70 ±1.20	<i>27.10 ±1.97</i>	25.30 ±1.84	Simple
	Linear Reg.	29.00 ±2.31	26.00 ±1.77*	29.25 ±3.08	26.30 ±2.00	TradSeg
	Logistic Reg.	25.70 ±2.08	24.70 ±2.08	25.95 ±1.94	26.30 ±2.51	Simple
Magic	MLP	15.16 ±0.53	14.84 ±0.60*	15.24 ±0.53	13.51 ±0.58*	FBTSeg
	Linear Reg.	32.03 ±2.81	18.73 ±0.56*	27.31 ±2.65*	16.06 ±0.65*	FBTSeg
	Logistic Reg.	20.98 ±0.44	18.29 ±0.67*	18.69 ±0.55*	15.40 ±0.51*	FBTSeg
Adult	MLP	14.89 ±0.91	14.71 ±0.96	13.60 ±1.33*	15.07 ±0.86	NNTree
	Linear Reg.	17.38 ±1.23	15.81 ±0.78*	13.68 ±0.83*	16.73 ±1.25	NNTree
	Logistic Reg.	14.90 ±1.00	14.96 ±0.97	14.58 ±0.97	14.96 ±0.82	Simple
Spambase	MLP	7.06 ±0.60	7.15 ±0.91	6.81 ±0.57	7.45 ±0.71	Simple
	Linear Reg.	24.36 ±6.83	9.02 ±0.92*	6.45 ±0.61*	10.61 ±0.75*	NNTree
	Logistic Reg.	8.69 ±0.63	8.67 ±0.58	6.76 ±0.66*	8.19 ±0.76	NNTree

In the German dataset, only the TradSeg method using linear regression significantly improved the results of the base classifier. Using MLP, the NNTree segmentation had significantly worse results. A possible explanation to such results is the small number of examples in this dataset. This fact leads to small segments in the model tree, and this situation is aggravated by the need of examples for validation. It is likely that this is the main reason why the segmentations did not improve the results, generating classifiers that are weaker than the ones of the original nodes.

In the Magic dataset, the segmentations improved the results of nearly all combinations. The linear regressions obtained the bigger gains; in particular, FBTSeg improved the average error from 32.03% to 16.06%. Using MLP, only the results of FBTSeg and TradSeg were significantly better than that of the base classifier. Using logistic regression, all three segmentations (NNTree, FBTSeg and TradSeg) improved the results, but only FBTSeg was statistically better than the base classifier. It is important to notice that, in this dataset, FBTSeg significantly improved the average error using all three techniques, when compared to the base classifiers, and also when compared to the other segmentations.

In the Adult dataset, all three segmentations improved the results of the base classifier using linear regression, but the FBTSeg results were not significantly better. The NNTree method was the only one to significantly improve the results using MLP and none of them obtained significant improvements using logistic regression.

In the Spambase dataset, using MLP, none of segmentation methods presented statistically significant results. With linear regression, they all significantly improved the base classifier, but the NNTree method was also noticeably better than the other segmentation methods. Using logistic regression, they all had smaller errors but only NNTree was significantly better than the base classifier.

4 Conclusion

This paper presented the results of experiments comparing three different segmentation methods, using three different base classifiers, and five UCI datasets. The traditional segmentation based on information gain and another two more recent segmentation methods (NNTree and FBTSeg) were tested against well-known classifiers such as linear regression, logistic regression, and MLP neural network.

Our results show that segmentation may be a viable alternative to improve the performance of classifiers. One of its advantages is that it may improve the results of older techniques, within the existing expertise of the modeling team. Even using linear regression, a classifier that is known to be weaker [25], the use of segmentation methods may lead to results that are similar and sometimes superior to those

obtained with newer and more predictive techniques such as MLP neural networks [25] [2]. Moreover, segmentations can also improve the results using MLP: in three of the five tested datasets, at least one of the three segmentation methods presented statistically better results.

From all the classifiers (simple or combined), the FBTSeg method achieved the best performance in the Chess and Magic datasets, obtaining 5 (five) of the possible 6 (six) best results, when we consider the three techniques available. The very best results in these two datasets were obtained with the FBTSeg + MLP combination.

Even though the NNTree method was not originally developed to work with the statistic regressions, it improved the results of these base classifiers in 7 (seven) of the possible 10 (ten) combinations; only in the German dataset it did not improve any results; and, perhaps surprisingly, it even achieved the best results in the Adult and Spambase datasets when combined with linear regression.

Although the TradSeg method is older and simple, it also improved the results of the base classifiers in many configurations. However, in general, both NNTree and FBTSeg presented better results. Nevertheless, none of the segmentation methods was clearly better than the others in all datasets. Thus, our conclusion is that it is worth testing more than one segmentation method to choose the one that is more suitable to the specific problem in hand.

As future work, it is important comparing the segmentation methods tested in this paper with other methods for combining classifiers such as Bagging [26], Boosting [26], and Stacking [4] [5] [26]. It is worth testing other base classifiers such as SVM [26] and k-NN [26] as well. Running experiments with other datasets is also recommended. Another possibility of future work is to experiment different split strategies for the segmentation tree, for example, using n -ary trees and/or different split algorithms.

5 References

- [1] O. P. Rud, "Data Mining Cookbook: Modeling Data for Marketing, Risk and Customer Relationship Management", John Wiley & sons, New York, 2001.
- [2] P. J. L. Adeodato, G. C. Vasconcelos, A. L. Arnaud, R. A. F. Santos, R. C. L. V. Cunha, and D. S. M. P. Monteiro, "Neural Networks vs. Logistic Regression: a Comparative Study on a Large Data Set", In proceedings of 17th International Conference on Pattern Recognition (ICPR'04), Volume 3, pp. 355-358, Cambridge, England, UK, 2004.
- [3] P. Cortez, A. Cerdeira, F. Almeida, T. Matos and J. Reis, "Modeling Wine Preferences by Data Mining from Physicochemical Properties", Decision Support Systems, Elsevier, Volume 47, Number 4, pp. 547-553, 2009.

- [4] D. H. Wolpert, "Stacked Generalization", *Neural Networks*, Volume 5, Number 2, pp. 241-259, 1992.
- [5] S. Dzeroski, and B. Zenko, "Is Combining Classifiers with Stacking Better than Selecting the Best One?" *Machine Learning*, Volume 54, Number 3, pp. 255-273, 2004.
- [6] G. Brown, J. Wyatt, R. Harris, and X. Yao, "Diversity Creation Methods: a Survey and Categorization", *Journal of Information Fusion*, Volume 6, Number 1, pp. 5-20, 2005.
- [7] T. Gestel, B. Baesens, P. Dijcke, J. Suykens, et al., "Linear and Non-linear Credit Scoring by Combining Logistic Regression and Support Vector Machines", *Journal of Credit Risk*, Volume 1, Number 4, 2006.
- [8] M. M. Islam, X. Yao, S. M. S. Nirjon, M. A. Islam, and K. Murase, "Bagging and Boosting Negatively Correlated Neural Networks", *IEEE Transaction on System, Man, and Cybernetics, Part B*, Volume 38, Number 3, pp.771-84, 2008.
- [9] S. L. Wang, K. Shafi, C. Lokan, and H. A. Abbass, "Robustness of Neural Ensembles Against Targeted and Random Adversarial Learning". In *proceedings of IEEE International Conference on Fuzzy Systems (FUZZ-IEEE)*, pp. 1-8, 2010.
- [10] K. W. D. Bock and D. V. D. Poel, "An Empirical Evaluation of Rotation-based Ensemble Classifiers for Customer Churn Prediction", *Expert Systems with Applications*, Volume 38, Number 10, pp 12293-12301, 2011.
- [11] O. Maimon, and L. Rokach, "Decomposition Methodology for Knowledge Discovery and Data Mining: Theory and Applications", *Series in Machine Perception and Artificial Intelligence*, Volume 61, World Scientific pub., 2005.
- [12] L. C. Thomas, D. Edelman, and J. Crook, "Credit Scoring and its Applications", *Society for Industrial and Applied Mathematics*, Philadelphia, USA, 2002.
- [13] C. Blake, and C. Merz, "UCI Repository of Machine Learning Databases", *School of Information and Computer Sciences*, University of California, Irvine, USA, 2012. <http://archive.ics.uci.edu/ml/> (Access 07/05/2012).
- [14] R. A. Johnson, and D. W. Wichern, "Applied Multivariate Statistical Analysis", *Prentice Hall*, Upper Saddle River, New Jersey-NJ, USA, sixth edition, 2007.
- [15] A. J. Macleod, "A Generalization of Newton-Raphson". *International Journal of Mathematical Education in Science and Technology*, Volume 15, Number 1, pp. 117-120, 1984.
- [16] J. Neter, M. H. Kutner, C. J. Nachtsheim, and W. Wasserman, "Applied Linear Statistical Models", *McGraw-Hill*, fourth edition, 1996.
- [17] S. Haykin, "Neural Networks and Learning Machines", third edition, *Prentice Hall*, New Jersey-NJ, USA, 2009.
- [18] H. Chipman, E. I. George, and R. E. McCulloch, "Bayesian Treed Models", *Machine Learning*, Volume 48, Numbers 1-3, pp. 299-320, 2002.
- [19] P. Maji, "Efficient Design of Neural Network Tree using a New Splitting Criterion". *Neurocomputing*, Volume 71, Numbers 4-6, pp. 787-800, 2008.
- [20] R. A. F. Santos, "Um Método para Segmentação de Preditores", *Doctorate Thesis*, Centro de Informática, Universidade Federal de Pernambuco, 2010. In Portuguese. <http://www.cin.ufpe.br/~roberto/AlunosPG/Teses/2010-PhD-Roberto.zip>.
- [21] W. J. Conover, "Practical Nonparametric Statistics", third edition. *John Wiley & sons*, 1999.
- [22] T. Fawcett, "An Introduction to ROC Analysis", *Pattern Recognition Letters*, Vol. 27, Number 8, pp. 861-874, 2006.
- [23] B. Zenko, L. Todorovski, and S. Dzeroski, "A Comparison of Stacking with MDTs to Bagging, Boosting, and Other Stacking Methods". In *proceedings of European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML/PKDD 2001)*, Workshop: Integrating Aspects of Data Mining, pp. 163-175, Freiburg, Germany, 2001.
- [24] SAS Institute Inc., "Finding the Solution to Data Mining: a Map of the Features and Components of SAS ® Enterprise Miner". *SAS Institute White Paper*, Cary, North Carolina, 2000.
- [25] M. Y. Kiang, "A Comparative Assessment of Classification Methods", *Decision Support Systems*, Vol. 35, Number 4, pp. 441-454, 2003.
- [26] I. H. Witten, and E. Frank, "Data Mining: Practical Machine Learning Tools and Techniques with Java Implementations", *Morgan Kaufmann*, San Francisco, 2005.